



POLITÉCNICA

EJERCICIO DE S. FICHEROS (TRANSPARENCIA 33)

Fernando Pérez Costoya
Curso 2026/27

nodo-i	permisos	tipo	usuario	grupo	enlaces	Localización
2	rwX r-x r-x	dir	root	root	<u>xxx</u>	100
3	rwX r-x r-x	dir	root	root	<u>yyy</u>	101
4	rwX rwX ---	dir	juan	graf	2	102
5	rwX r-x ---	dir	blanca	graf	2	103
6	rwX r-x ---	dir	irene	prog	2	104
7	rwX r-s ---	fich	juan	graf	1	107,108
8	rw- rw- rw-	fich	juan	graf	1	106
9	rw- --- ---	fich	blanca	graf	<u>zzz</u>	105

Agrup 100		Agrup 101		Agrup 102		Agrup 103		Agrup 104		Agrup 105		Agrup 106	
.	2	.	3	.	4	.	5	.	6	1234567890		123abc456def	
..	2	..	2	..	3	..	3	..	3				
home	3	juan	4	ejecutor	7	fich2.txt	9	dat.txt	21				
tmp	15	blanca	5	fich.txt	8	new	12	real	18				
bin	37	irene	6			increm.dat	19	fich2.txt	9				

Responda a las siguientes preguntas

- a) (i) Indica los valores xxx yyy y zzz de la primera tabla del sistema de ficheros.
(ii) Pinta la estructura del árbol de ficheros/directorios resultante.

inodo raíz

	nodo-i	permisos	tipo	usuario	grupo	enlaces	Localización	
1	2	rwX r-x r-x	dir	root	root	xxx	100	2
5	3	rwX r-x r-x	dir	root	root	yyy	101	6
9	4	rwX rwX ---	dir	juan	graf	2	102	10
17	5	rwX r-x ---	dir	blanca	graf	2	103	18
23	6	rwX r-x ---	dir	irene	prog	2	104	24
13	7	rwX r-s ---	fich	juan	graf	1	107,108	
15	8	rw- rw- rw-	fich	juan	graf	1	106	
21	9	rw- --- ---	fich	blanca	graf	zzz	105	

3		7		11		19		25					
Agrup 100		Agrup 101		Agrup 102		Agrup 103		Agrup 104		Agrup 105		Agrup 106	
.	2	.	3	.	4	.	5	.	6	1234567890		123abc456def	
..	2	..	2	..	3	..	3	..	3				
home	3	juan	4	ejecutor	7	fich2.txt	9	dat.txt	21				
tmp	15	blanca	5	fich.txt	8	new	12	real	18				
bin	37	irene	6			increm.dat	19	fich2.txt	9				

Vamos a recorrer en profundidad el árbol del sistema de ficheros

DIBUJO PASO A PASO DE ÁRBOL DE FICHEROS (PASO 1)

dati © UPM 2022



directorio
raíz



DIBUJO PASO A PASO DE ÁRBOL DE FICHEROS (PASO 2)

dati © UPM 2022

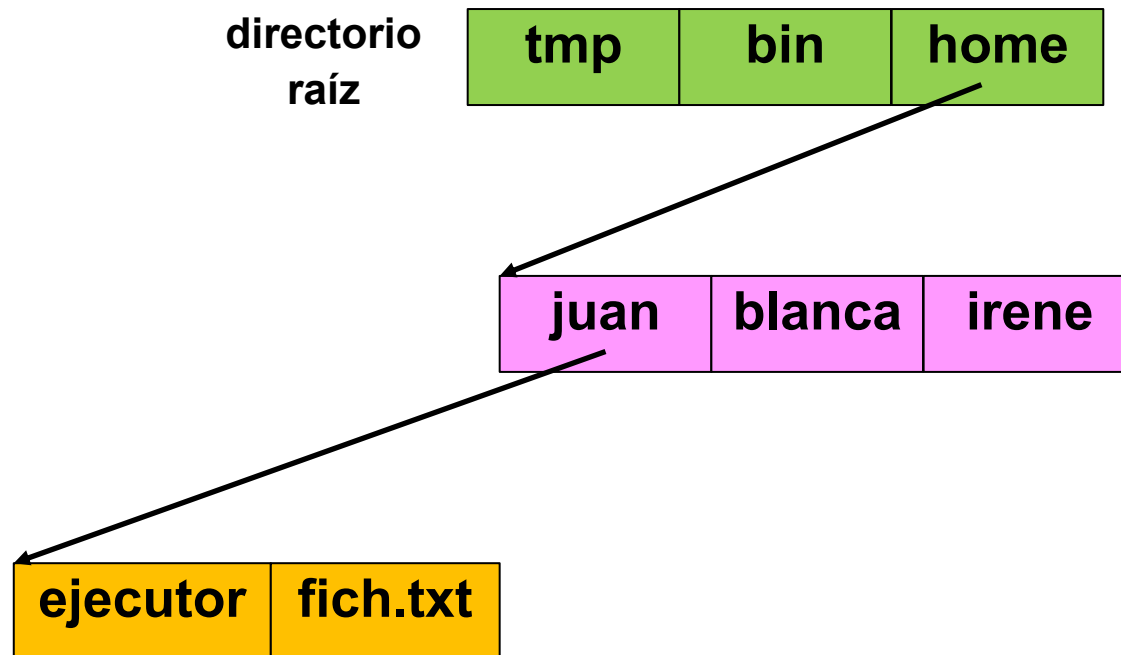


directorio
raíz



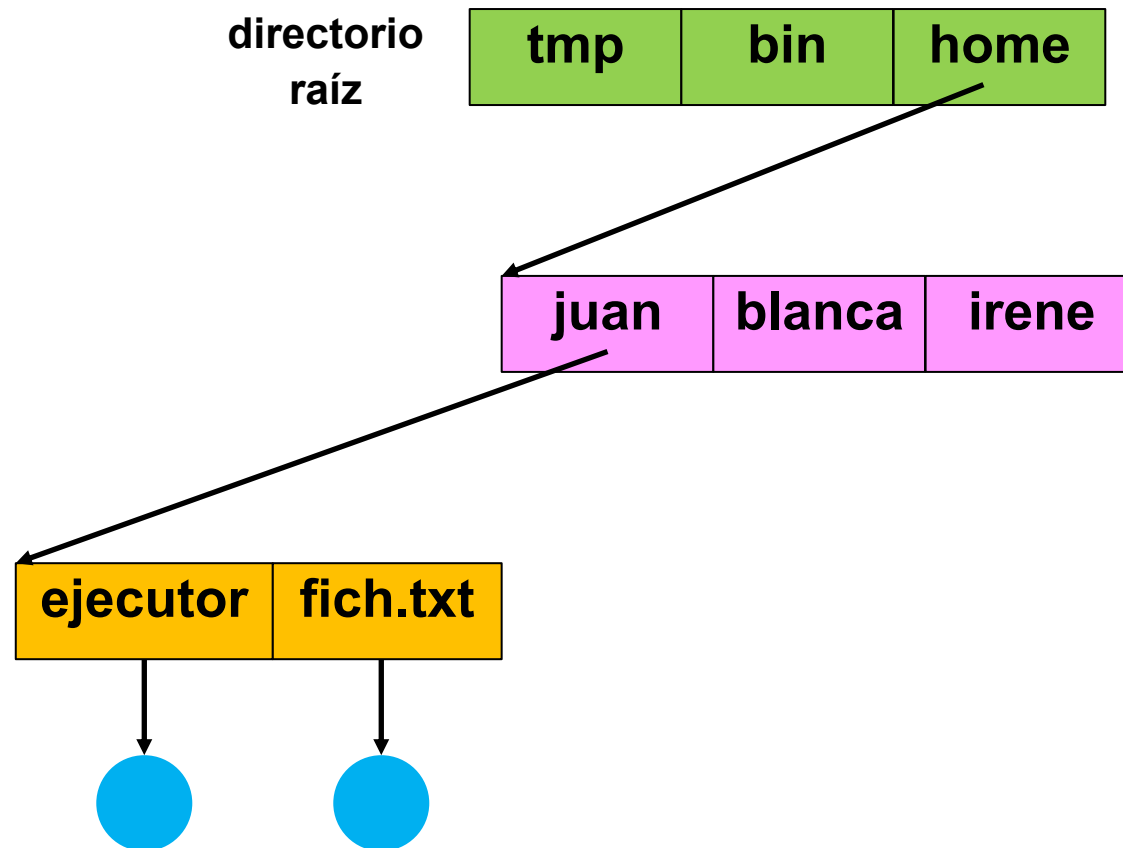
DIBUJO PASO A PASO DE ÁRBOL DE FICHEROS (PASO 3)

dati © UPM 2022



DIBUJO PASO A PASO DE ÁRBOL DE FICHEROS (PASO 4)

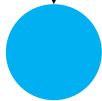
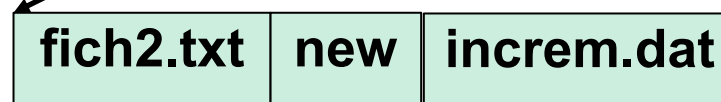
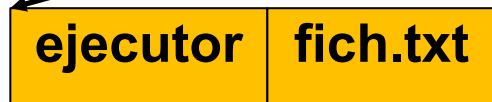
dati © UPM 2022



DIBUJO PASO A PASO DE ÁRBOL DE FICHEROS (PASO 5)

dati © UPM 2022

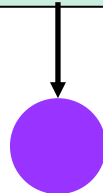
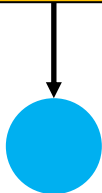
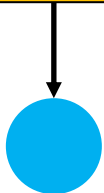
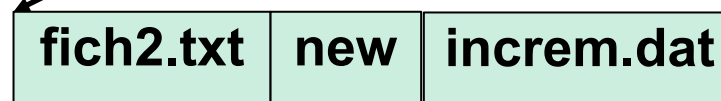
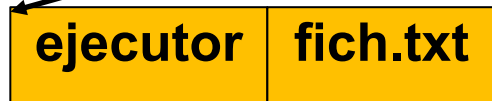
directorio
raíz

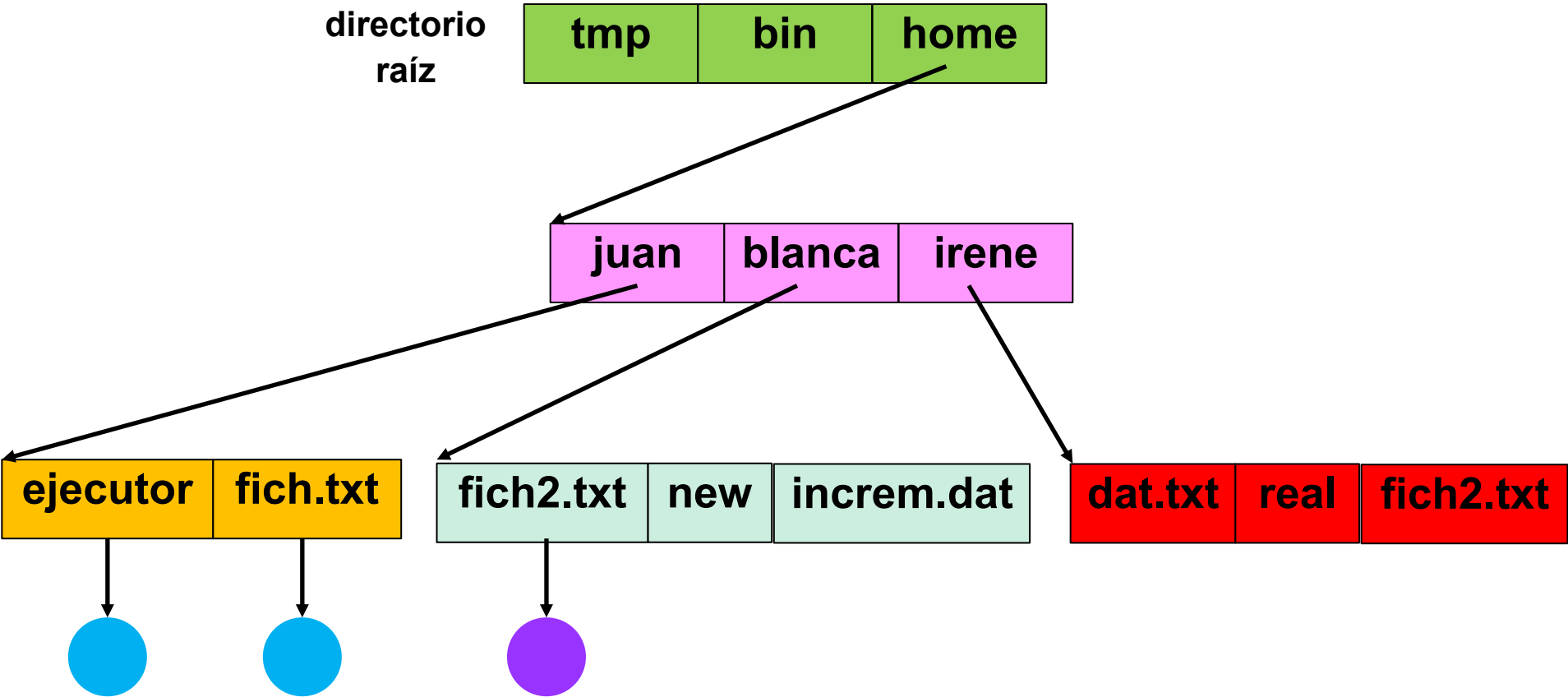


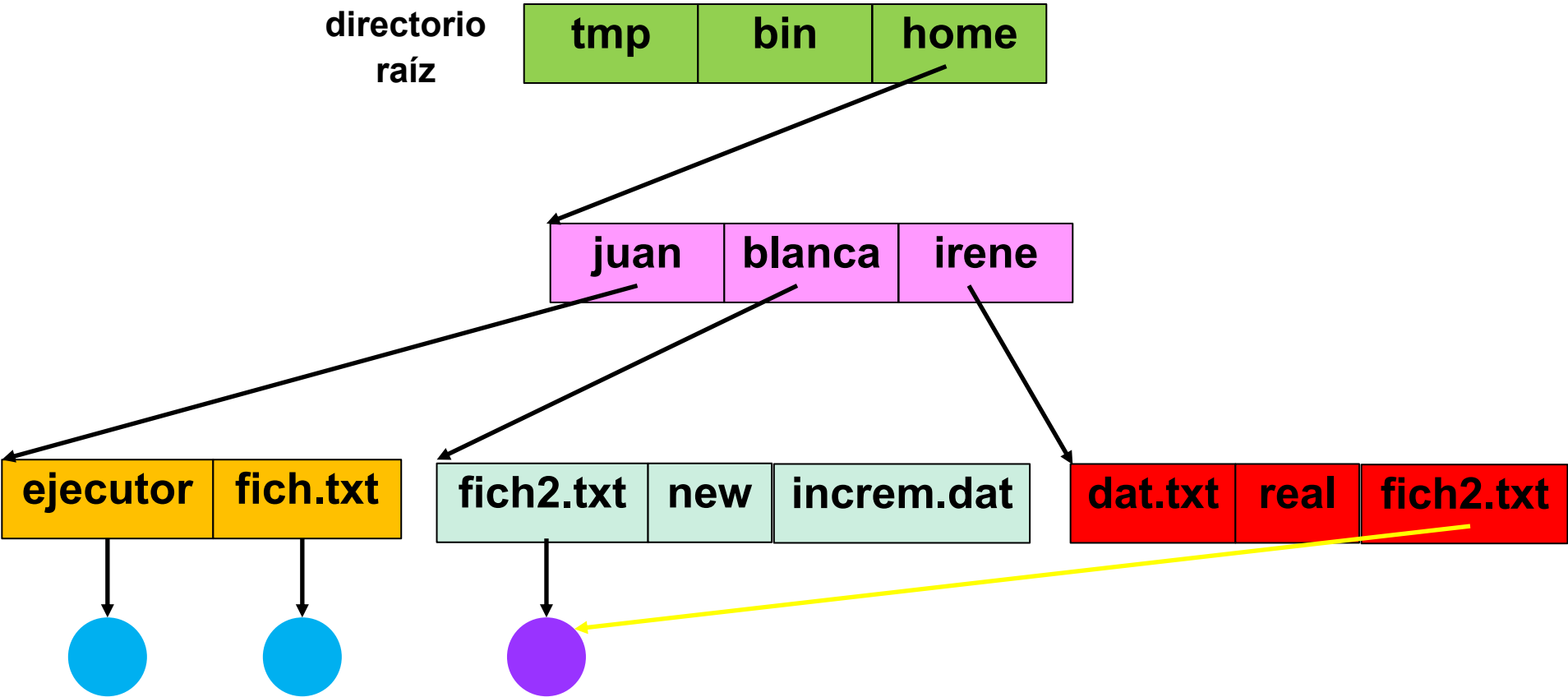
DIBUJO PASO A PASO DE ÁRBOL DE FICHEROS (PASO 6)

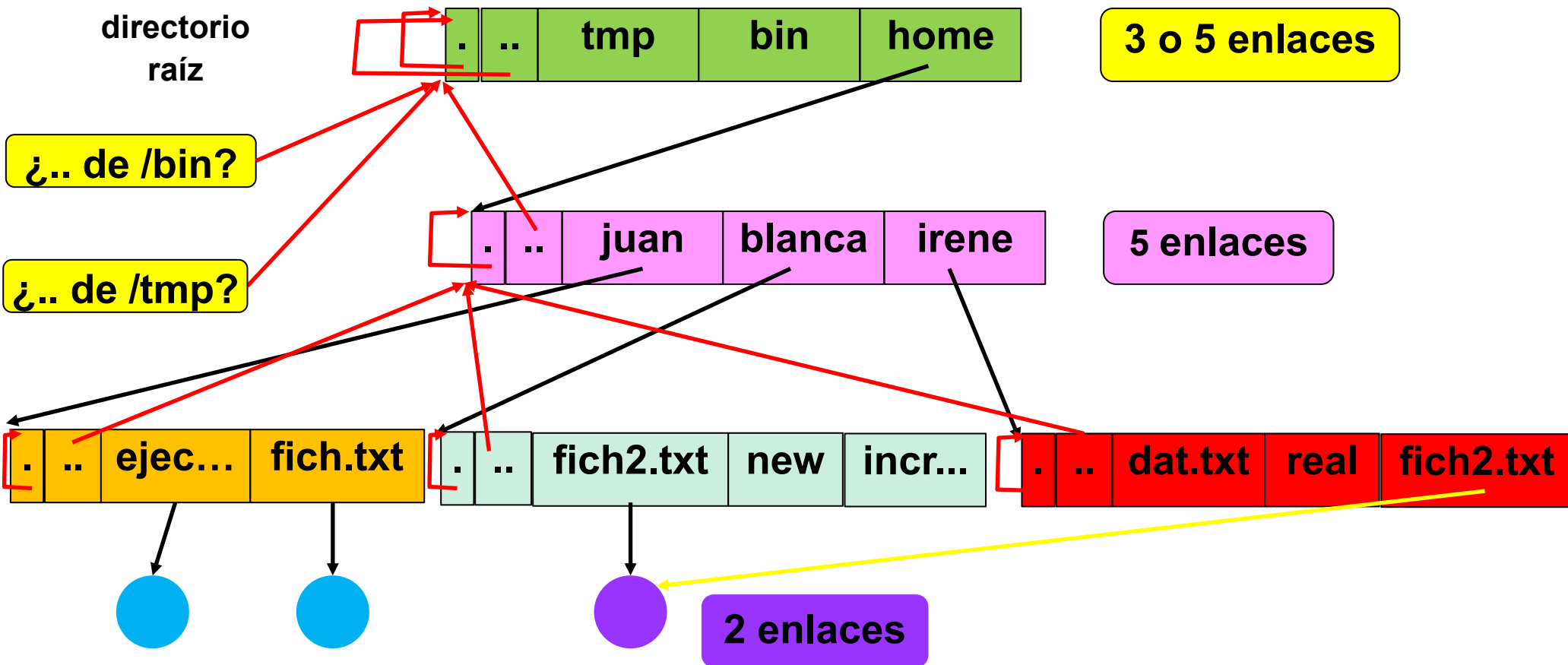
dati © UPM 2022

directorio
raíz









REGLA: nº enlaces directorio = 1 por nombre + 1 por `.` + 1 por el `..` de cada subdirectorio
 Para directorio raíz: 1 por `.` + 1 por su propio `..` + 1 por el `..` de cada subdirectorio

NOTA: No sabemos si `/tmp` y `/bin` son directorios: valoramos las dos opciones

ENUNCIADO (APARTADO (b))

datos © UPM 2022



nodo-i	permisos	tipo	usuario	grupo	enlaces	Localización
2	rwX r-X r-X	dir	root	root	3 o 5	100
3	rwX r-X r-X	dir	root	root	5	101
4	rwX rwX ---	dir	juan	graf	2	102
5	rwX r-X ---	dir	blanca	graf	2	103
6	rwX r-X ---	dir	irene	prog	2	104
7	rwX r-s ---	fich	juan	graf	1	107,108
8	rw- rw- rw-	fich	juan	graf	1	106
9	rw- --- ---	fich	blanca	graf	2	105

depende de si
/bin y /tmp son dir.

Agrup 100		Agrup 101		Agrup 102		Agrup 103		Agrup 104		Agrup 105	Agrup 106
.	2	.	3	.	4	.	5	.	6	1234567890	123abc456def
..	2	..	2	..	3	..	3	..	3		
home	3	juan	4	ejecutor	7	fich2.txt	9	dat.txt	21		
tmp	15	blanca	5	fich.txt	8	new	12	real	18		
bin	37	irene	6			increm.dat	19	fich2.txt	9		

b) Considere que el usuario *blanca* (del grupo *graf*) invoca, desde su home, el siguiente mandato:

/home/juan/ejecutor ./fich2.txt

- (i) ¿Podrá ponerse en ejecución dicho mandato?
- (ii) ¿Qué identidad tendrá el proceso resultante?

Permisos requeridos para que *blanca* del grupo *graf* ejecute */home/juan/ejecutor*:

- Atravesar directorio raíz / necesita permiso **x**:
 - Usuario no es dueño del directorio raíz ni de su grupo
 - Se aplican los permisos para el resto: `rwxr-xr-x` → **tiene permiso**
- Atravesar directorio */home* necesita permiso **x**:
 - Usuario no es dueño de ese directorio ni de su grupo
 - Se aplican los permisos para el resto: `rwxr-xr-x` → **tiene permiso**
- Atravesar directorio */home/juan* necesita permiso **x**:
 - Usuario no es dueño de ese directorio pero sí es de su grupo
 - Se aplican los permisos para el grupo: `rwxrwx---` → **tiene permiso**
- Ejecutar el fichero */home/juan/ejecutor* necesita permiso **x**:
 - Usuario no es dueño de ese directorio pero sí es de su grupo
 - Se aplican los permisos para el grupo: `rwxr-s---` → **tiene permiso**
 - NOTA: en notación simbólica esa “s” implica “x” para el grupo y SETGID (2750)

Identidad del proceso resultante:

- Ejecución de fichero con SETGID: GID efectivo = GID del dueño del fichero
- UID efectivo (= real): *blanca*; GID efectivo (= GID fichero): *graf*

SOLUCIÓN (APARTADO (b))



(blanca,graf) exec
/home/juan/ejecutor
✓

Ejecutable SGID
UID real = blanca
UID efect. = blanca
GID real = graf
GID efectivo = graf

nodo-i	permisos	tipo	usuario	grupo	enlaces	Localización
2	rwX r-X r-X	dir	root	root	3 o 5	100
3	rwX r-X r-X	dir	root	root	5	101
4	rwX rwx ---	dir	juan	graf	2	102
5	rwX r-X ---	dir	blanca	graf	2	103
6	rwX r-X ---	dir	irene	prog	2	104
7	rwX r-S ---	fich	juan	graf	1	107,108
8	rw- rw- rw-	fich	juan	graf	1	106
9	rw- --- ---	fich	blanca	graf	2	105

Agrup 100		Agrup 101		Agrup 102		Agrup 103		Agrup 104		Agrup 105	Agrup 106
.	2	.	3	.	4	.	5	.	6	1234567890	123abc456def
..	2	..	2	..	3	..	3	..	3		
home	3	juan	4	ejecutor	7	fich2.txt	9	dat.txt	21		
tmp	15	blanca	5	fich.txt	8	new	12	real	18		
bin	37	irene	6			increm.dat	19	fich2.txt	9		

ENUNCIADO (APARTADO (c))



datos © IIBM 2022

nodo-i	permisos	tipo	usuario	grupo	enlaces	Localización
2	rwX r-X r-X	dir	root	root	3 o 5	100
3	rwX r-X r-X	dir	root	root	5	101
4	rwX rwX ---	dir	juan	graf	2	102
5	rwX r-X ---	dir	blanca	graf	2	103
6	rwX r-X ---	dir	irene	prog	2	104
7	rwX r-s ---	fich	juan	graf	1	107,108
8	rw- rw- rw-	fich	juan	graf	1	106
9	rw- --- ---	fich	blanca	graf	2	105

Agrup 100		Agrup 101		Agrup 102		Agrup 103		Agrup 104		Agrup 105		Agrup 106	
.	2	.	3	.	4	.	5	.	6	1234567890		123abc456def	
..	2	..	2	..	3	..	3	..	3				
home	3	juan	4	ejecutor	7	fich2.txt	9	dat.txt	21				
tmp	15	blanca	5	fich.txt	8	new	12	real	18				
bin	37	irene	6			incrm.dat	19	fich2.txt	9				

c) El usuario irene del grupo prog ejecuta el siguiente mandato:

rm /home/irene/fich2.txt

Describa, paso a paso, cómo se realizaría en el sistema de ficheros

Permisos requeridos para que *irene* del grupo *prog* borre */home/irene/fich2.txt*:

- Atravesar directorio raíz / necesita permiso **x**:
 - Usuario no es dueño del directorio raíz ni de su grupo
 - Se aplican los permisos para el resto: **rw****xr**-**xr**-**x** → **tiene permiso**
- Atravesar directorio */home* necesita permiso **x**:
 - Usuario no es dueño de ese directorio ni de su grupo
 - Se aplican los permisos para el resto: **rw****xr**-**xr**-**x** → **tiene permiso**
- Atravesar directorio */home/irene* necesita permiso **x**:
 - Usuario es dueño de ese directorio
 - Se aplican los permisos para el dueño: **rw****xr**-**x**--- → **tiene permiso**
- Borrar fichero */home/irene/fich2.txt* necesita permiso **w** en directorio que lo contiene
 - Usuario es dueño de ese directorio
 - Se aplican los permisos para el dueño: **rw****xr**-**x**--- → **tiene permiso**
 - **NOTA:** puede borrarse el fichero aunque no sea suyo ni tenga ningún permiso
 - Borrado de fichero solo requiere permiso de escritura en directorio que lo contiene

Operaciones sobre el SF:

- Eliminar la entrada *fich2.txt* en el directorio *irene*
- Decrementar nº de enlaces en el inodo ($2 \rightarrow 1$): como $\neq 0$ el fichero sigue existiendo

SOLUCIÓN (APARTADO (c))



(irene,prog) **rm**
/home/irene/fich2.txt

**rm requiere “w”
en directorio;
da igual permisos
de fichero**

nodo-i	permisos	tipo	usuario	grupo	enlaces	Localización
2	rwX r-X r-X	dir	root	root	3 o 5	100
3	rwX r-X r-X	dir	root	root	5	101
4	rwX rwX ---	dir	juan	graf	2	102
5	rwX r-X ---	dir	blanca	graf	2	103
6	rwX r-X ---	dir	irene	prog	2	104
7	rwX r-s ---	fich	juan	graf	1	107,108
8	rw- rw- rw-	fich	juan	graf	1	106
9	rw- --- ---	fich	blanca	graf	1 - 2	105

Agrup 100		Agrup 101		Agrup 102		Agrup 103		Agrup 104		Agrup 105	Agrup 106
.	2	.	3	.	4	.	5	.	6	1234567890	123abc456def
..	2	..	2	..	3	..	3	..	3		
home	3	juan	4	ejecutor	7	fich2.txt	9	dat.txt	21		
tmp	15	blanca	5	fich.txt	8	new	12	real	18		
bin	37	irene	6			increm.dat	19	fich2.txt	9		

- d) El usuario blanca invoca el mandato **/home/juan/ejecutor ./fich2.txt** desde su home.
Parte del código del programa ejecutor es el siguiente:

Proceso	código ejecutado
ejecutor	<pre>umask(0027); fd = creat(argv[1], 0666); n = write(fd, "0123456789ABCDEFGHJK", 20); fd2 = open(argv[1], O_RDWR); n = write(fd2, "0123456789", 5);</pre>

- (i) ¿Qué sucede en la llamada creat? ¿Con qué permisos se queda el fichero fich2.txt?
- (ii) Indique la situación en la que se encuentran las tablas internas del sistema operativo relacionadas con la gestión de ficheros (tablas de descriptores, tabla intermedia y tabla de copia de nodos-i) después de la ejecución del código descrito, suponiendo que no se produce error alguno.
- (iii) Implemente el código necesario, para que, escribiendo un único byte adicional en el fichero pasado como primer argumento al programa (argv[1]), el tamaño del mismo se quede en 10000 bytes. ¿Cuántas agrupaciones ocuparía el fichero después de esta operación?

(i) ¿Qué sucede en la llamada creat? ¿Con qué permisos se queda el fichero fich2.txt?

El fichero ya existe por lo que se mantienen los permisos y se trunca su contenido:

600 → rw-----

9	rw- --- --	fich	blanca	graf	1	0
---	------------	------	--------	------	---	---

Tablas en memoria

BCP

fd	
0	2
1	3
2	4
3	0
4	0
5	0
6	0
...	

Tabla intermedia

	Nodo-i	Posición	Referen.	rw
1				
2				
3				
4				
5	0	0	0	0
6	0	0	0	0
	0	0	0	0
	0	0	0	0

Tabla de inodos en memoria

Nodo-i	nopens	
.....		
.....		
.....		
.....		
0	0	0
0	0	0
0	0	0
0	0	0
0	0	0

fich2.txt
inodo 9 (1234567890)

¿después de creat(argv[1], 0666)?

Tablas en memoria

BCP

fd

0	2
1	3
2	4
3	5
4	0
5	0
6	0
...	

Tabla intermedia

	Nodo-i	Posición	Referen.	rw
1				
2				
3				
4				
5	9	0	1	01
6	0	0	0	0
	0	0	0	0
	0	0	0	0

Tabla de inodos en memoria

Nodo-i	nopens	
.....		
.....		
.....		
.....		
9	1	
0	0	0
0	0	0
0	0	0

fich2.txt
inodo 9

¿después del write de 20 bytes con descriptor 3?

Tablas en memoria

BCP

fd	
0	2
1	3
2	4
3	5
4	0
5	0
6	0
	...

Tabla intermedia

	Nodo-i	Posición	Referen.	rw
1				
2				
3				
4				
5	9	20	1	01
6	0	0	0	0
	0	0	0	0
	0	0	0	0

Tabla de inodos en memoria

Nodo-i	nopens	
.....		
.....		
.....		
.....		
9	1	
0	0	0
0	0	0
0	0	0

fich2.txt
inodo 9 0123456789ABCDEFGHIJK

¿después de open(argv[1], O_RDWR)?

Tablas en memoria

BCP	
fd	
0	2
1	3
2	4
3	5
4	6
5	0
6	0
	...

Tabla intermedia				
	Nodo-i	Posición	Referen.	rw
1				
2				
3				
4				
5	9	20	1	01
6	9	0	1	11
	0	0	0	0
	0	0	0	0

Tabla de inodos en memoria

Nodo-i	nopens	
.....		
.....		
.....		
.....		
9	2	
0	0	0
0	0	0
0	0	0

fich2.txt
inodo 9 0123456789ABCDEFGHIJK

¿después del write de 5 bytes con descriptor 4?

Tablas en memoria

BCP	
fd	
0	2
1	3
2	4
3	5
4	6
5	0
6	0
	...

Tabla intermedia			
	Nodo-i	Posición	Referen. rw
1			
2			
3			
4			
5	9	20	1 01
6	9	5	1 11
	0	0	0 0
	0	0	0 0

Tabla de inodos en memoria

Nodo-i	nopens	
.....		
.....		
.....		
.....		
9	2	
0	0	0
0	0	0
0	0	0

fich2.txt (0123456789ABCDEFGHIJK)
inodo 9

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
```

```
int main(int argc, char *argv[]) {
    int fd;
    char byte = 0;
```

```
    if (argc!=2) {
        fprintf (stderr, "Uso: %s archivo\n", argv[0]);
        return 1;
    }
```

```
    // Abre el archivo
```

```
    if ((fd=open(argv[1], O_WRONLY))<0) {
        perror("No puede abrirse el archivo");
        return 1;
    }
```

```
    lseek(fd, 9999, SEEK_SET);
```

```
    // sería mejor así (no gasta espacio adicional) pero no lo hemos estudiado
    //ftruncate(fd, 10000);
```

```
    if (write(fd, &byte, 1)!=1) {
        perror("error actualizando fichero");
        return 1;
    }
```

```
    close(fd);
    return 0;
}
```

El fichero generado al final del apartado d.ii tenía 1 agrupación. Después de ejecutar este programa, tendrá una 2ª agrupación